

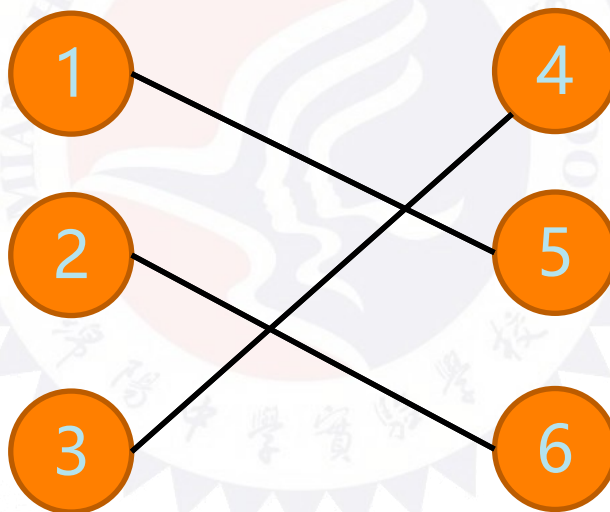
二分图算法及应用

主讲：黄凤林



→ 基本概念

- ✧ 设 $G=(V,E)$ 是一个无向图，如果顶点 V 可分割为**两个互不相交的子集** (X,Y) ，并且图中的**每条边** (i,j) **所关联的两个顶点** i **和** j **分别属于这两个不同的顶点集** $(i \in X, j \in Y)$ ，则称图 G 为一个二分图



例一 完美的牛栏

✧ 题目描述

有N头奶牛，M个牛栏，每头奶牛都只愿意在她们喜欢的那些牛栏中产奶。一个牛栏只能容纳一头奶牛，一头奶牛只能在一个牛栏中产奶。

计算最大分配方案。

✧ 数据范围： $N \leq 100$ ， $M \leq 100$ 。

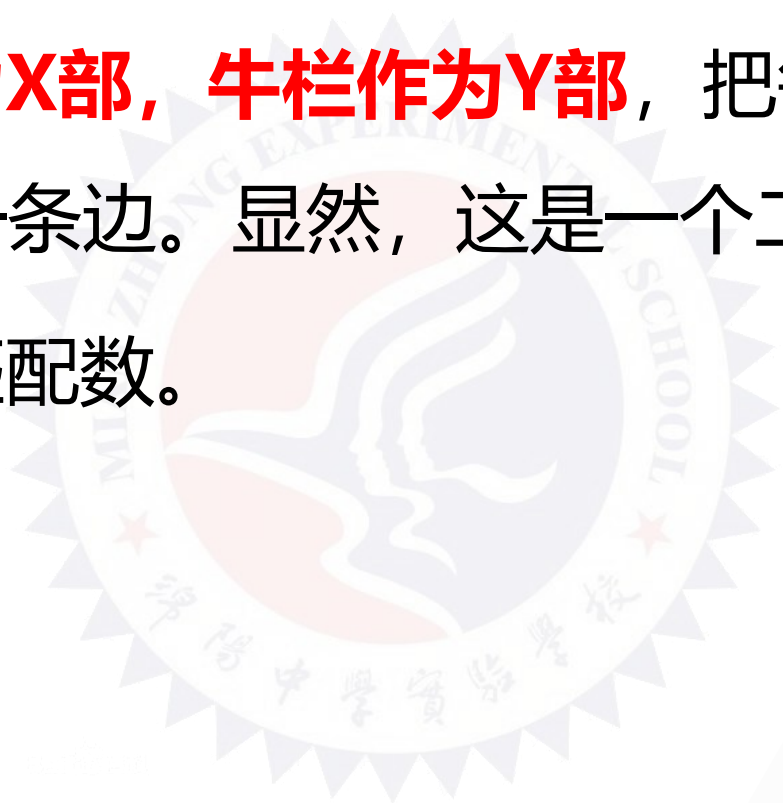


➡ 例一 完美的牛栏

✧ 思路分析

把**奶牛作为X部**，**牛栏作为Y部**，把每头奶牛和喜欢的牛栏连一条边。显然，这是一个二分图。

计算最大匹配数。





→ 匹配

- ✧ 给定一个二分图 G ，在 G 的一个子图 M 中， **M 边集中的任意两条边都不依附于同一个顶点**，则称 M 是一个匹配。
- ✧ 最大匹配：**边数最多的匹配**（子图 M ）称为图 G 的最大匹配。
- ✧ 完全匹配/完备匹配：如果一个匹配中，图中的**每个顶点都和图中某条边相关联**，则称此匹配为完全匹配，也称作完备匹配。

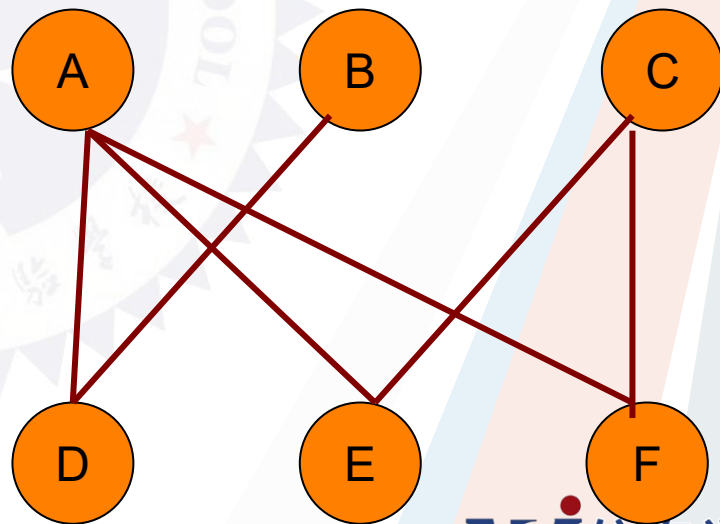
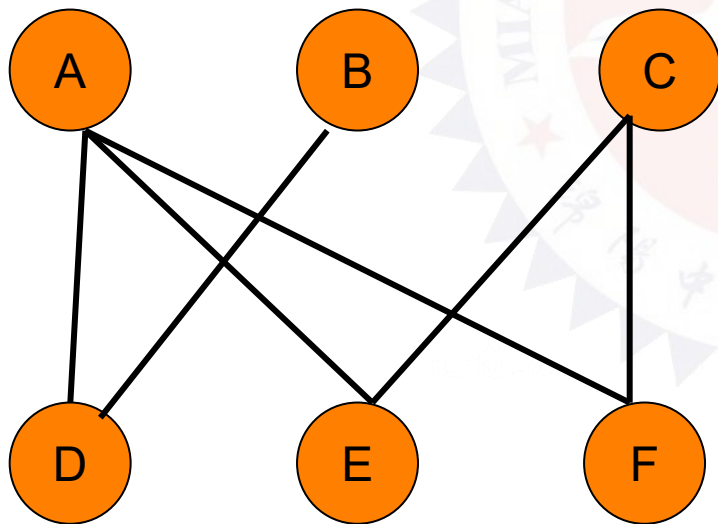


→ 匈牙利算法

- ✧ 算法的思路是不停的找增广路,并增加匹配的个数。
- ✧ 当不能再找到增广路时,就得到了一个最大匹配。
- ✧ 算法流程:
 - 1、置M为空
 - 2、找出一条增广路径P,通过取反操作获得更大的匹配M' 代替M
 - 3、重复2操作直到找不出增广路径为止

→ 增广路

- ✧ 若 P 是图 G 中一条连通两个未匹配顶点的路径,并且属 M 的边和不属 M 的边(即已匹配和待匹配的边)在 P 上交替出现,则称 P 为相对于 M 的一条增广路径。
- ✧ 增广路的四个特性:
 1. P 的路径长度必定为奇数, 第一条边和最后一条边都不属于 M 。
 2. P 上所有奇数边都不在 M 中, 偶数边都在 M 中。
 3. P 经过取反操作可以得到一个更大的匹配 M' , 何为“取反”。
 4. M 为 G 的最大匹配当且仅当不存在相对于 M 的增广路径。



➔ 找增广路的算法步骤

- ✧ “从点A出发的增广路径”一定首先连向一个在原匹配中没有与点A配对的点B。如果点B在原匹配中没有与任何点配对，则它就是这条增广路径的终点；反之，如果点B已与点C配对，那么这条增广路径就是从A到B，再从B到C，再加上“从点C出发的增广路径”。并且，这条从C出发的增广路径中不能与前半部分的增广路径有重复的点。
- ✧ 如果从一个点A出发，没有找到增广路径，那么无论再从别的点出发找到多少增广路径来改变现在的匹配，从A出发都永远找不到增广路径。

→ 匈牙利算法的实现

```
using namespace std;
const int maxn=200+10;
const int maxm=100000+10;
int n,m,ans=0; //n表示奶牛数量, m表示牛棚数量
int head[maxn];
struct node{
    int to,next;
} edge[maxm*2];
bool vis[maxn]; //当前匹配中访问的点标记
int link[maxn]; //link[y]=x表示当前与y相匹配的节点x
int cnte=0; //边集数量
void add(int u,int v){
    edge[++cnte].to=v;
    edge[cnte].next=head[u]; head[u]=cnte;
}
```



→ 匈牙利算法的实现

```
int init(){
    int x,y;
    memset(link,-1,sizeof(link));
    scanf("%d%d",&n,&m);
    for(int i=1;i<=n;i++){
        scanf("%d",&x);
        for(int j=1;j<=x;j++){
            scanf("%d",&y); add(i,y);
        }
    }
}

int main(){
    init();
    for(int i=1;i<=n;i++){
        memset(vis,false,sizeof(vis));
        if(find(i,0)) ans++;
    } printf("%d\n",ans);
    return 0;
}
```

→ 匈牙利算法的实现

```
bool find(int u,int fa){//找u点的增广路
    for(int i=head[u];i;i=edge[i].next){
        int v=edge[i].to;
        if(v==fa) continue;
        if(!vis[v]){//v点当轮匹配未访问过
            vis[v]=true;//访问节点v
            if(link[v]==-1 || find(link[v],u))
                {//如果v点未被匹配或从v点出发有增广路
                    link[v]=u;
                    return true;
                }
        }
    }
    return false;//未找到增广路
}
```

→ 算法分析

- ✧ 算法的核心是找增广路径的过程DFS
- ✧ 对于每个可以与 u 匹配的顶点 v ，假如它未被匹配，可以直接用 v 与 u 匹配；
- ✧ 如果 v 已与顶点 w 匹配，那么只需调用 $\text{dfs}(w)$ 来求证 w 是否可以与
其它顶点匹配，如果 $\text{dfs}(w)$ 返回 true 的话，仍可以使 v 与 u 匹配；
如果 $\text{dfs}(w)$ 返回 false ，则检查 u 的下一个邻接点.....
- ✧ 在 dfs 时，要标记访问过的顶点 ($\text{visit}[j]=\text{true}$)，以防死循环和
重复计算；每次在主过程中开始一次 dfs 前，所有的顶点都是未
标记的。
- ✧ 主过程只需对每个 X 部的顶点调用 dfs ，如果返回一次 true ，就
对最大匹配数加一；一个简单的循环就求出了最大匹配的数目。

时空分析

✧ 时间复杂度:

找一次增广路径的时间为:

邻接矩阵: $O(n^2)$

邻接表: $O(n+m)$

✧ 总时间:

邻接矩阵: $O(n^3)$

邻接表: $O(nm)$

✧ 空间复杂度:

邻接矩阵: $O(n^2)$

邻接表: $O(m+n)$

→ URAL 1109 - Conference

✧ 问题描述

一次会议，A国派出M个代表，B国派出N个代表。A国和B国的代表之间，有K对成员是相互可以进行交谈，每一个参加会议的代表至少包含在其中一对关系中。大会的CEO想在代表的房间建立直接的电话联系，要求每个人都至少跟对方的一个人能电话联系。每一个电话联系是在有结对关系的代表之间建立的。

问题：CEO想建立最少的电话联系的多少？

$N \leq 1000$, $M \leq 1000$

→ URAL 1109 - Conference

✧ 思路分析

一条电话线覆盖两个代表，最多有最大匹配条电话线互不相交。那么剩下的代表都需要至少搭建一条电话线。

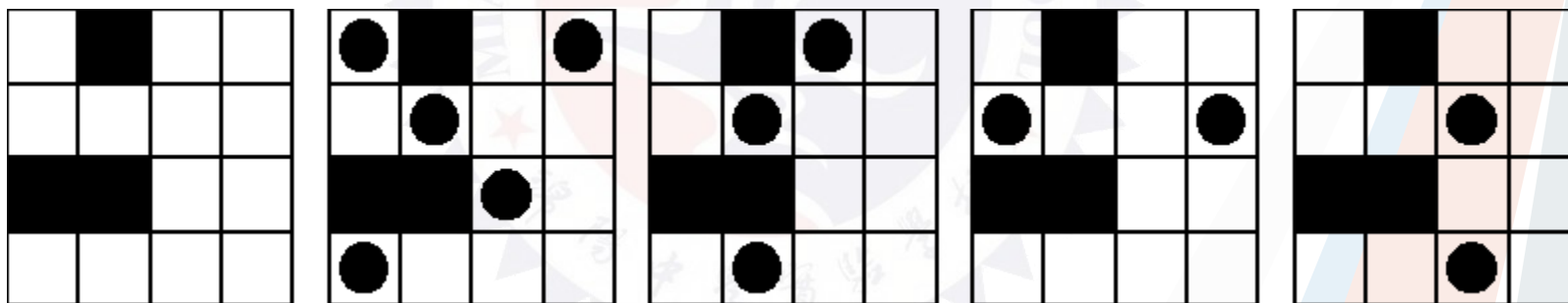
故有：最少的电话线 = $N + M - \text{最大匹配数}$

→ zoj 1002 - 火力网

✧ 题目描述

输入一个N ($N \leq 20$)，和N*N的棋盘布局，放入守卫，要求同行同列不能放置，如果遇到障碍物则可以放置，问最多可以放多少守卫。

如第一个图为棋盘布局，第2个图为放置5个守卫（最多），第3个图合格但不是最多的，第4 5个图不合格。





→ zoj 1002 - 火力网

✧ 思路分析

建图利用行连通块和列连通块做点，一个守卫能管所在行联通块和所在列联通块。

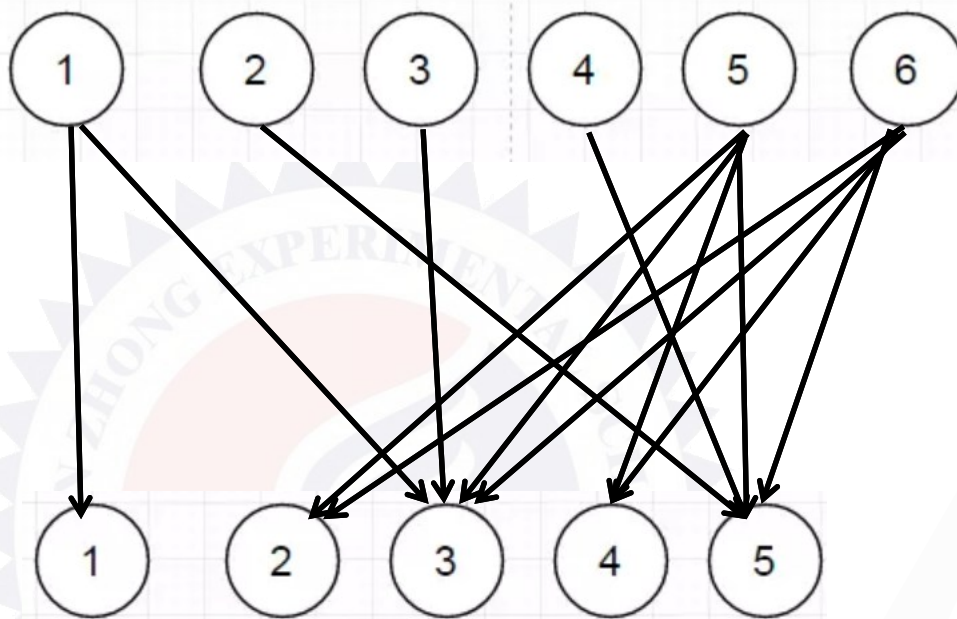
要保证每个守卫所管联通块不能相交，那么将棋盘中的格子对应二分图中的边。

求最大匹配即可。

z oj 1002 -火力网

1		5	6
1	3	5	6
		5	6
2	4	5	6

1		2	2
3	3	3	3
		4	4
5	5	5	5



于是我们的方法就出来了，将所有的横块与竖块编上号，如果它们相交便连上边。最后只需要求出最大匹配数即可。

➔ POJ 1325 – 任务安排

✧ 题目描述

有两台机器A和B及N个需要运行的任务。

每台机器有M种不同的模式，而每个任务i都恰好在一台机器上运行。如果它在机器A上运行，则机器A需要设置为模式 a_i ，如果它在机器B上运行，则机器B需要设置为模式 b_i 。每台机器上的任务可以按照任意顺序执行，但是每台机器每转换一次模式需要重新启动一次。

请合理为每个任务安排一台机器并合理安排顺序，使得机器重启次数尽量少。

$N \leq 100$, $M \leq 1000$

→ POJ 1325 – 任务安排

✧ 思路分析

将A机器的每个模式看成一个X结点，B机器的每个模式看成一个Y结点，任务i为边 (a_i, b_i) ，构造二分图。

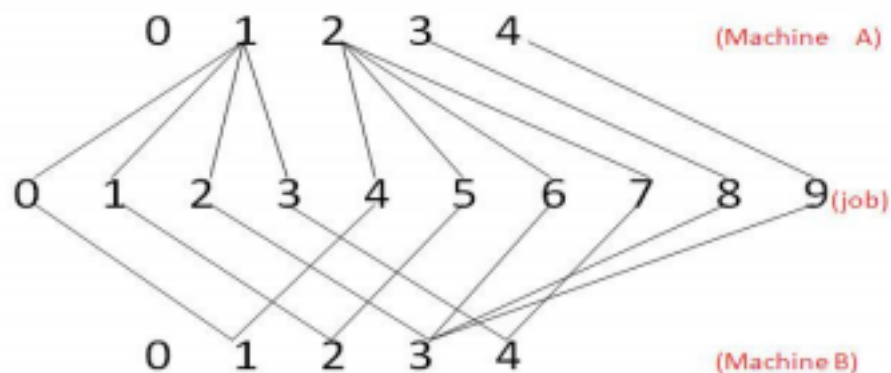


图 (1)

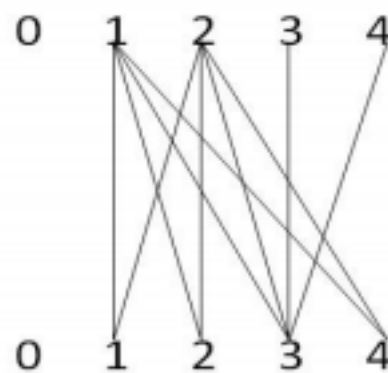


图 (2)

→ POJ 1325 – 任务安排

转变为图(2)之后发现，之前的job一定在图2中的某一条边上，同一条边上可以包含多个job，所以要完成所有的job，只需要包含所有的边即可。

本题即为**选最少的点让每条边都至少与其中一个点关联**(X部或Y部均可)，这就是二分图的**最小点覆盖**问题。

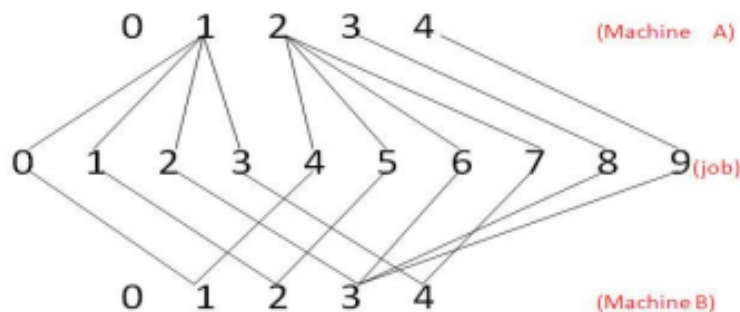


图 (1)

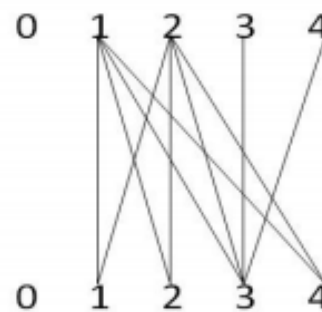


图 (2)

➔ 最小点覆盖

✧ 最小点覆盖：

寻找一个点集(X 集合或 Y 集合的都行)，使得图中每条边都至少有一个点在该点集中。

要求这个点集的结点数最少。

✧ 可以证明：

最少的点（即覆盖数） = 最大匹配数

➔ 如何构造一组最小覆盖

- 求得最大匹配 M 后，图中已经不存在交错路。从右边未匹配的点开始，以寻找交错路的方式扩展节点，并标记路过的节点。
- 取右边未标记的节点，左边标记的节点，则构成一组最小覆盖。
- 证明：所取节点数 $=|M|$ 。右边未匹配的点肯定被标记了，左边未匹配的点不可能被标记（否则找到了交错路），一条匹配边的左端点被标记，那么右端点也肯定被标记，所以所取节点恰覆盖一条匹配边。所取的点覆盖了所有边。假设存在一条边未被覆盖，则该边右端点被标记了，左端点未标记。如果这不是一条匹配边，那么相当于找到了交错路，如果着是一条匹配边，那么右端点只能通过其匹配的左端点到达。

→ POJ 2226 - Muddy fields

✧ 题目描述:

在一块 $R \times C$ 的地面上，有一些格子是泥泞的，现在要用一些宽为1的木板把泥地盖住，并且不能盖住好地。

木板可以重叠。问最少需要多少木板？ $R, C \leq 50$

Sample Input

4 4

```
* *.
. *.
***
.
***
.
*
..
```

Sample Output

4

铺放方法

1.2.

.333

444.

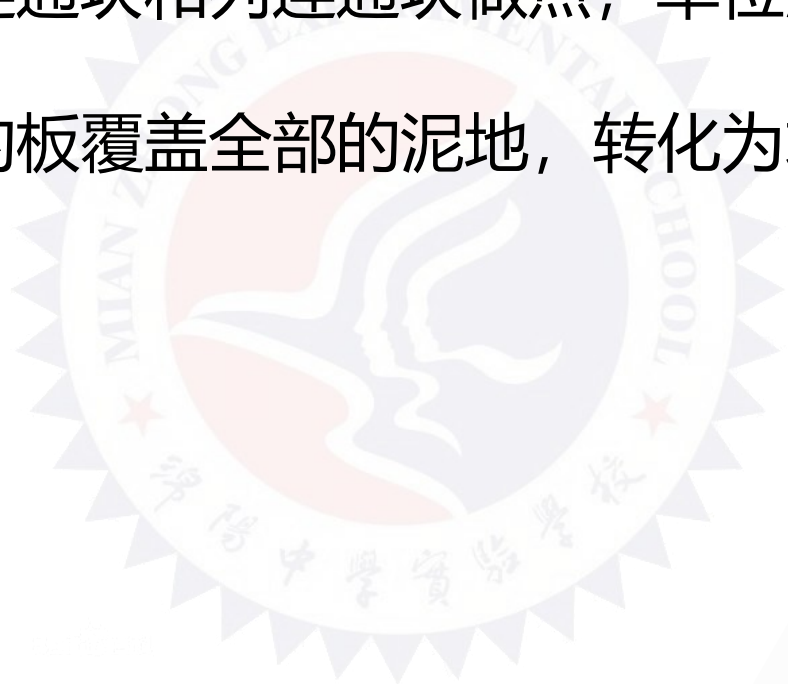
..2.



→ POJ 2226 - Muddy fields

✧ 思路分析:

最小覆盖是覆盖所有的边，因此泥地对应边。
建图利用行连通块和列连通块做点，单位泥地对应二分图中的边。
要求放最少的板覆盖全部的泥地，转化为求最小覆盖





→ PKU 1422 - Air Raid

✧ 题目描述

给定一个N个点的有向图，在这个图上的某些点上放伞兵，伞兵可以走到图上所有的点，且每个点只被一个伞兵走一次。

问至少放多少伞兵？

✧ 数据范围： $N \leq 120$



→ PKU 1422 - Air Raid

✧ 思路分析

由于每个点只能被一个伞兵走一次，所以一个伞兵走的是一条有向路径。

本题即是求：用最少的路径，覆盖图中所有的点。

最小路径覆盖



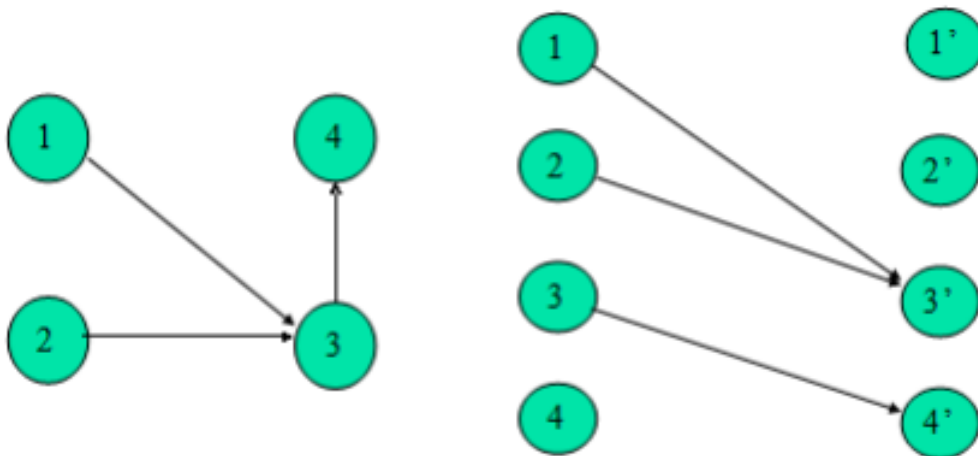
→ DAG最小路径覆盖

✧ DAG最小路径覆盖

用**尽量少的不相交简单路径覆盖有向无环图 G 的所有结点。**

解决此类问题可以建立一个二分图模型，把所有顶点 i 拆成两个集合： X 集合集中 i ； Y 集合集中的 i' ，如果有边 $i \rightarrow j$ ，则在二分图中引入边 $i \rightarrow j'$ 。

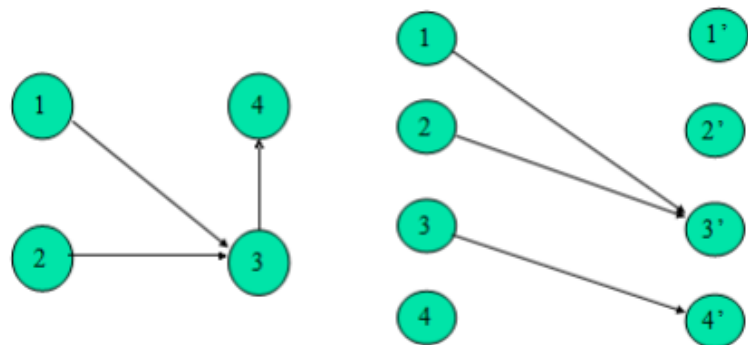
设二分图最大匹配为 m ，则结果就是 $n-m$ 。



→ 最小路径覆盖

图中，对应左边的DAG建立构造右边的二分图，可以找到二分图的一个最大匹配 M ：1-3'，3-4'，那么 M 中的这两条匹配边怎样对应DAG中的路径的边？

使二分图中一条边对应DAG中的一条有向边，(1-3')对应DAG图中的有向边1- $\rightarrow$ 3，这样DAG中1就会有一个后继顶点（3是1的唯一后继，因为二分图中一个顶点至多关联一条边），所以1不会成为DAG中一条路径中的结尾顶点，同样，(3-4')对应DAG中3- $\rightarrow$ 4，3也不会成为结尾顶点，那么原图中总共4个顶点，减去2个有后继的顶点(1,3)，就剩下没有后继的顶点(2,4)，即DAG路径的结尾顶点，而每个结尾顶点正好对应DAG中的一条路径，二分图中寻找最大匹配 M ，就是找到了对应DAG中的非路径结尾顶点的最大数目，那么**DAG中顶点数 - $|M|$** 就是DAG中结尾顶点的最小数目，即DAG的最小路径覆盖数。



→ POJ 2594 - Treasure Exploration

✧ 题目描述

给你一个有向无(有)环图，有 n 个点， m 条有向边。

一个机器人可以从任意点开始沿着边的方向走下去。对于每一个机器人：走过的点不能再走过。

问你最少用几个机器人可以走完所有的 n 个点，**不同的机器人可以走相同的点。**

✧ 数据范围： $n \leq 500$ ， $m \leq 5000$

→ POJ 2594 - Treasure Exploration

✧ 思路分析

由于每一个机器人不能走重复的点，所以一个机器人走的是
一条有向路径。

本题即是求：**允许路径有重叠顶点的最小路径覆盖。**

具体做法：允许路径有**重叠顶点的最小路径覆盖**

在做最小路径覆盖之前做一次**传递闭包**即可，原理如下：

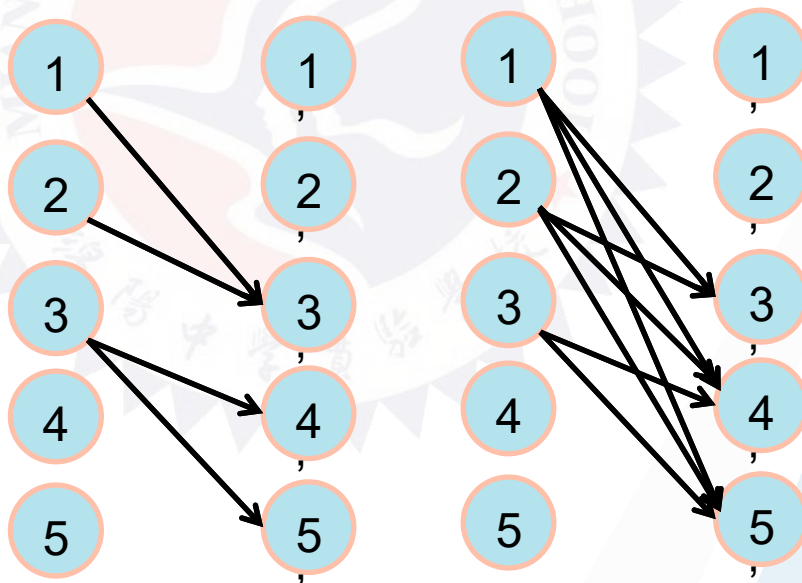
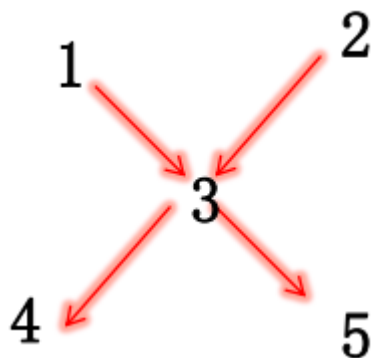
设两条路径 p 和 q 有重(chong)点 $i[1], i[2], \dots, i[n]$ ，易知 $i[1]$ 和 $i[n]$ 不是 p, q 两路径的起点和终点，设 p 有子路径 $u \rightarrow i[1] \rightarrow i[2] \rightarrow \dots \rightarrow i[n] \rightarrow u'$ ， q 有子路径 $v \rightarrow i[1] \rightarrow i[2] \rightarrow \dots \rightarrow i[n] \rightarrow v'$ ，则传递闭包后增添弧 $\langle v, v' \rangle$ ，则将 q 的子路径 $v \rightarrow i[1] \rightarrow i[2] \rightarrow \dots \rightarrow i[n] \rightarrow v'$ 改为 $v \rightarrow v'$ ，即能保证 p, q 之间没有重点。

POJ 2594 - Treasure Exploration

允许路径有重叠顶点的最小路径覆盖

如果直接求最大匹配，得到2，用顶点数减后，得到结果是3。这显然不符合3可以重复走的条件。

在做传递闭包后，增加1-4， 1-5， 2-4， 2-5四条边，然后匹配数为3，用顶点数减后，得结果为2。这就正确了。



→ 传递闭包

✧ 传递闭包的定义:

对于有向图 $G(V, E)$ 的传递闭包即是 $G(V, E)$, **修正图的完整性**
其中 $E\{(i, j): \text{图} G \text{ 中包含一条由 } i \text{ 到 } j \text{ 的路径}\}$ 。

在多个顶点的有向图中, 每个顶点可以到按照方向到达一定的节点, 这叫图的连通性。有种方法直接告诉我们, 图中的两个节点是否可以联通, 这里说的是WarShall算法。

WarShall的基本原理是, **如果A可以到达B, 且C可以到达A, 则C可以到达B**。通过对邻接矩阵的修正可以做到这点。随然这里举例是将两步可并成一步, 但数学上可以证明这种修正可以达到任意步骤。

→ 传递闭包算法实现一

由于只需要确定节点对 (i,j) 之间是存在 $i \rightarrow j$ 的路径，所以，对于松弛操作可以有两种优化方式，

(1)将所有节点对之间的存在的直接连通的边权重设为1，不连通设为INF（无穷大）。然后运行该算法，如果 $mp[i][j] < n$;则 (i, j) 之间存在一条简单路径。如果 $mp[i][j] = INF$,则两者之间不存在路径。

(2)可以将松弛策略改为， $mp[i][j] == 1 \parallel (mp[i][k] == 1 \ \&\& \ mp[k][j] == 1)$ 也即是要么 i 可以到达 j ,要么 i 可以通过 $\{1,2,3,...,k-1\}$ 中的部分节点到达 j 。

Floyd warshall 传递闭包算法的实现，时间复杂度为 $O(V^3)$;

```
void floyd_warshall(){
    for(int k = 1; k <= n; ++k)
        for(int i = 1; i <= n; ++i)
            for(int j = 1; j <= n; ++j)
                if(mp[i][j]==1 || mp[i][k]==1 && mp[k][j]==1)
                {
                    mp[i][j] = 1;
                }
}
```

DFS 传递闭包算法

算法分析:

DFS时间复杂度为 $O(V+E)$;

使用对每个节点进行DFS,每次可以得出一个节点的可以到达的节点,可以求出有向图的传递闭包,时间复杂度为 $V*O(V+E)$,即 $O(V*(V+E))$;

如果图的边数较少时,第二种算法更有效,可以根据题目的数据约束进行选择。

```
void vis[500+10][500+10]; //vis[i][j]表示i->j可到达
void dfs(int s,int u){
    for(int i=head[u];i;i=edge[i].next){
        int v=edge[i].to;
        vis[s][v]=1;
        dfs(s,v);
    }
}
//调用:
for(int i=1;i<=n;i++) dfs(s,s);
```

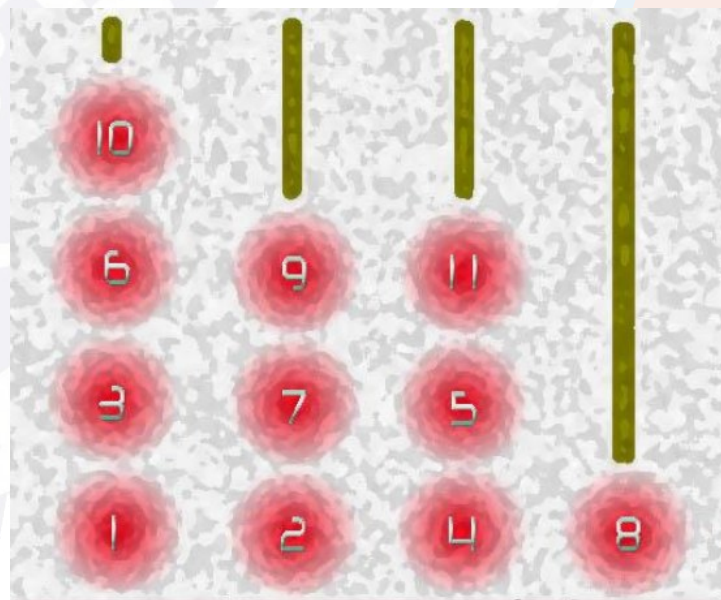
→ ZOJ 1239 - Hanoi Tower Troubles Again!

✧ 题目描述

给定柱子数 N ，按编号从小到大放球，要求：如果该球不在最底部，则该球和它下面一个球的编号之和必须为完全平方数。

问最多能放多少球上去。

✧ 数据范围： $N \leq 50$



→ ZOJ 1239 - Hanoi Tower Troubles Again!

✧ 思路分析

这题首先需要转化一下，将问题从求N根柱子最多能放多少球;变为放K个球最少需要多少柱子?

每个球看做是一个节点，对于球 $a, b (a < b)$ ，如果 $a + b$ 是完全平方数，那么有一条 a 指向 b 的边，由于 a 总是小于 b ，显然所建图无环。

每根柱子相当于一条路径，而且每个球只能位于一根柱子上，所以保证了路径不会相交，至此，问题转化为**最小路径覆盖**。

能够放的球的数目是随着柱子数单调递增的。根据答案的单调性，我们可以通过二分答案解，将每次二分出的答案和N做比较，最后得出答案。

→ POJ 1466 - Girls and Boys

✧ 题目描述

学校里有男生和女生一个 n 人，其中某些人之间是浪漫关系（一男一女）。现在要求找出一个最大的集合，使得里面的任意两个人都没有浪漫关系。

✧ 数据范围： $n \leq 500$

Sample Input 1

```
7
0: (3) 4 5 6
1: (2) 4 6
2: (0)
3: (0)
4: (2) 0 1
5: (1) 0
6: (2) 0 1
```

Sample Output 1

5

Sample Input 2

```
3
0: (2) 1 2
1: (1) 0
2: (1) 0
```

Sample Output 2

2

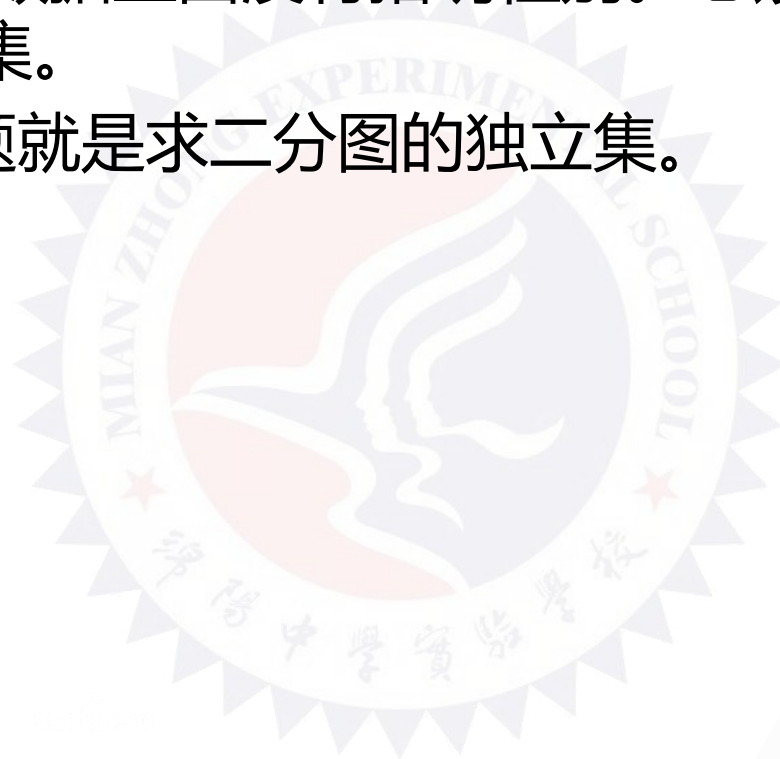


→ POJ 1466 - Girls and Boys

✧ 例题分析

首先，输入数据里面没有指明性别。必须先进行染色，区分出X集和Y集。

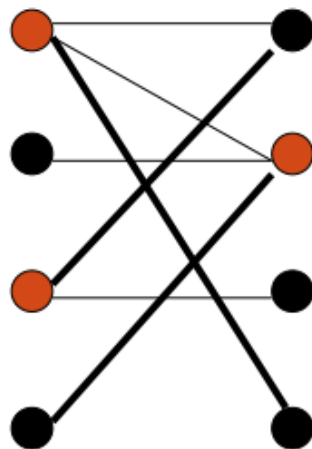
容易看出，问题就是求二分图的独立集。





→ 最大独立集

- ✧ 在 N 个点的图 G 中选出 m 个点，使这 m 个点两两之间没有边，求 m 最大值。
- ✧ **如果** 图 G 满足二分图条件，则可以用二分图匹配来做。
- ✧ **最大独立集点数 = N - 最大匹配数。**



黑色点即为一个最大独立集

例二 骑士问题

❖ 题目描述

一个 $n \times n$ 的棋盘上,有一些单位小方格不能放置骑士。
求出最多可以放置几个相互不攻击的骑士。

X代表攻击范围, S代表骑士

❖ 数据范围: $n \leq 20$

	X		X	
X				X
		S		
X				X
	X		X	



例二 骑士问题

思路分析

对棋盘染色，设方格的坐标为 (x,y) ， x 和 y 同奇偶的方格对应X集合，不同奇偶的对应Y集合。由于骑士沿着“日”字形路线攻击，所以每个攻击肯定是处于X集合和Y集合之间，而不可能在两个集合内部。

显然，转化后变为求二分图的最大独立集。



SGU 234 - Black-White King Strikes Back

✧ 题目描述

给定 n, m ($1 \leq n, m \leq 200$) 和一个 $n \times m$ 的矩阵。

矩阵由0和1组成，0表示障碍，1表示可以放守卫。

问题是要求一种放置守卫的方案使得守卫最多（如多解任意输出一种）。

守卫放置必须满足任意2个守卫不能相邻（上下左右）。



SGU 234 - Black-White King Strikes Back

输出第一行包括1个整数——最多守卫数。
接下来 $n*m$ 矩阵, "#"表示障碍, "G"表示守卫,
"."表示空地。

Input

```
3 5
1 1 1 0 0
1 0 0 0 1
1 1 0 1 1
```

Output

```
5
G.G##
.###G
G.#G.
```



SGU 234 - Black-White King Strikes Back

✧ 思路分析

对矩阵黑白染色，矩阵中的1则分为两个集合，且相邻两个1属于不同集合。对于相邻的1连边。
显然，转化为求最大独立集及方案。



➔ 二分图最大匹配的应用

变种1：二分图的最小顶点覆盖

最小顶点覆盖:要求用最少的点（X或Y中都行），让每条边都至少和其中一个点关联。

knoig定理：最小顶点覆盖数 = 二分图的最大匹配数(m)

变种2： DAG图的最小路径覆盖

用尽量少的不相交简单路径覆盖有向无环图(DAG)G的所有顶点，这就是DAG图的最小路径覆盖问题。

结论：DAG图最小路径覆盖数=节点数(n)-最大匹配数(m)

变种3:二分图的最大独立集

结论：二分图的最大独立集数=节点数(n)- 最大匹配数(m)



→ 作业

ZOJ 1654 - Place the Robots

PKU 2942 - Knights of the Round Table

拓展：最佳匹配KM算法

参考文献：

2002年冬令营论文 - 孙方成 《偶图的算法及应用》

2004年冬令营论文 - 黄源河 《浅谈图论模型的建立与应用》

NOI2016

The background features a central point from which several rays of varying colors (blue, orange, red, and grey) extend outwards. Faint binary code (0s and 1s) is scattered across the background, and a small line graph with two lines (one blue, one orange) is visible on the right side.

Thank you!